1. Project Brief

Mission

Build **Capital Command**: a private, AI-assisted personal financial operating system that securely consolidates bank activity, cash flow, budgets, goals, investments, market intelligence, documents, and conversational analysis into one dashboard.

Target User / ICP

- Primary user: you—a technically advanced U.S.-based operator/investor who needs daily visibility into working capital, personal and business-adjacent cash flow, investment exposure, and upcoming obligations.
- V1 tenancy: single-user, single household, with an architecture that can later support advisors, entities, and small teams.
- Primary trigger: "What is my true cash position, what changed today, what is due next, and what financial action needs attention?"

Pain Points

- Bank, credit, brokerage, crypto, goals, bills, and market research live in separate systems.
- Working-capital visibility is delayed because transactions require manual reconciliation and categorization.
- Investment research is fragmented across pricing feeds, SEC filings, Reddit, and X.
- Tax, employment, and legal documents are hard to find, incomplete, and disconnected from financial events.
- Generic AI chat tools do not have permissioned, cited access to the user's own financial ledger.

Success Metrics and Constraints

Area	V1 target	Constraint
Transaction freshness	Webhook-triggered sync plus reconciliation every 6 hours	Plaid data availability varies by institution
Categorization quality	≥90% correct after user feedback	Never silently overwrite user rules
Daily cash clarity	Show available cash, 7/30/90-day runway, upcoming obligations	Separate cleared, pending, and forecast values

Area	V1 target	Constraint
Alert reliability	≥99% successful delivery logged and retryable	Alerting must be idempotent
Financial close	Monthly P&L generated in <5 minutes after review	Requires explicit personal/business allocation rules
AI quality	Every numeric claim traceable to internal ledger rows	No trade execution or tax/legal conclusions
Delivery	V1 in 6–8 weeks; private beta	Start with free/low-cost APIs and existing Plaid access
Budget	$0–$75/month excluding Plaid production usage and LLM usage	Sandbox is free; live Plaid access is not an unlimited free production service. [1]

Scope Boundaries

- **In V1:** read-only bank and investment connectivity, transaction sync, budgeting, goals, P&L, notifications, market dashboard, social/news ingestion, document vault, document extraction, and LLM chat.
- **Not in V1:** executing equity/options/crypto trades, automated tax filing, legal advice, credit decisions, multi-tenant public launch, custody of funds, or storing brokerage/order credentials.
- Options content must be strictly informational and risk-labeled. Options can expire worthless, and uncovered options can create substantial or potentially unlimited loss exposure. [2] [3]

2. Product Requirements Document

Leadership Summary

Project: Capital Command
Owner: You
Phase: PreD → D1 Design
Status: Feasible, with security and data-governance design as the release gate.

What it is: A secure personal finance and investment intelligence dashboard that turns banking, spending, documents, and market signals into an explainable daily operating view. It centralizes working capital while preserving user control over every financial classification and recommendation.

Why now: You already have Plaid access and a modern Vercel/Supabase/GitHub stack; the highest-value first move is a read-only, private system before expanding into broker or tax workflows.

Success looks like:

- Daily bank activity appears with clear pending/posted status and actionable alerts.

- You can view cash flow, category spend, goals, P&L, positions, and market context from one dashboard.
- An LLM can answer ledger-grounded questions with citations and cannot initiate money movement or trades.

Risks / blockers: Plaid production entitlement and pricing confirmation, personal-vs-business accounting policy, data encryption/key management, X API cost/access, document sensitivity, and the final definition of "real-time."

System Overview

Core Functional Mechanics

1. **Connect and synchronize**
    - Create a Plaid Link session on demand.
    - Exchange the public token server-side only; encrypt the resulting `access_token`.
    - Run an initial `transactions/sync` import, then save the cursor per Plaid Item.
    - Receive `SYNC_UPDATES_AVAILABLE`, validate the webhook, enqueue a delta sync, classify new rows, update balances, evaluate alerts, and notify.
    - Plaid's Transactions flow supports incremental sync via `/transactions/sync`; it also supports webhooks for available sync updates and transaction lifecycle events. [4] [5]
2. **Compute financial truth**
    - Maintain immutable raw transaction records and a separate normalized ledger.
    - Maintain three views: posted cash, pending cash, and forecast cash.
    - Create user-reviewed rules for merchant normalization, transfer detection, reimbursements, business allocation, and recurring expenses.
    - Generate cash flow, P&L, budget variance, net-worth snapshots, and runway from the normalized ledger.
3. **Track investments and markets**
    - Retrieve user-authorized investment holdings and investment transactions where available through Plaid Investments. Plaid exposes holdings and up to 24 months of investment transactions for supported investment accounts, with update webhooks for holdings and investment transaction changes. [6] [7]
    - Use Alpaca's market-data APIs for equity, option, and crypto prices; its documentation supports real-time and historical data across equities, options, and crypto. [8] [9]
    - Use the SEC's unauthenticated EDGAR JSON endpoints for company submissions and XBRL company facts. [10] [11]
    - Treat oil, natural gas, commodities, and broad macro data as a **later adapter**, initially sourced through market ETFs/futures proxy tickers and user-entered watchlists—not a trading feed.
4. **Ingest and analyze information**

- Ingest user-selected RSS feeds directly.
- Use Reddit OAuth for designated communities and watchlists. Reddit's official data API requires authenticated access for normal usage and documents a free-use limit of 100 queries per minute per OAuth client ID for eligible access. [12]
- Treat X as optional: its API supports recent-post search and filtered streaming, but it is not assumed to be a free V1 dependency. [13] [14]
- Extract tickers, themes, sentiment, entities, claims, source links, and confidence—not trading instructions.

5. **Documents and forms**
   - Upload, OCR/extract, tag, search, and summarize tax, employment, legal, and financial documents.
   - Link documents to transactions, entities, calendar dates, deductions, and tasks.
   - Provide structured tax-prep exports and prefill assistance, but route actual filing to IRS-approved/official workflows. IRS Free File Fillable Forms are intended for taxpayers comfortable preparing their own forms and instructions. [15] [16]
6. **Chat with financial context**
   - Use retrieval over user-authorized ledger rows, documents, goals, budgets, market snapshots, and approved social/news artifacts.
   - Require every answer to distinguish:
     - calculated facts,
     - forecast assumptions,
     - market/community signals,
     - educational explanation.
   - Return supporting transaction, document, market, or source links with every materially financial answer.

Architectural Pattern

- **Frontend:** Next.js App Router dashboard with server components for protected reads and client components for charts, feeds, and live state.
- **Backend:** Next.js route handlers / server actions for synchronous requests; background worker for webhooks, syncs, document parsing, and alerting.
- **Data:** Supabase Postgres as system of record; Supabase Storage for encrypted documents; Redis-compatible queue/cache as a worker dependency.
- **Orchestration:** ROSTR Hub with PAL-compiled policies, NPAO gating, RAG DAL adapters, and ContextEngine project memory.
- **Event model:** append-only external-event inbox → idempotent worker → normalized domain tables → materialized metrics → alert evaluator.
- **Security model:** least privilege, read-only financial integrations, encrypted secrets, row-level isolation, audit log, explicit confirmation for any future write action.

Website Map

Route	Purpose	Main capabilities
`/`	Daily Command Center	Working capital, cash forecast, alerts, daily activity, market pulse
`/accounts`	Connected accounts	Balances, account health, connection status, reconnect flow
`/transactions`	Ledger	Search, filters, split/recategorize, attachment link, rules
`/cash-flow`	Liquidity	Historical inflow/outflow, burn, recurring bills, forecast
`/budget`	Plan vs actual	Envelopes, category budgets, variance, rollovers
`/goals`	Goals	Savings/debt/investment goals, contribution plans, scenarios
`/pnl`	Personal P&L	Monthly income, deductible/business expenses, allocations, close workflow
`/portfolio`	Holdings	Allocation, P&L, concentration, account-level positions
`/markets`	Markets	Watchlists, equity/options/crypto cards, SEC filings, macro proxies
`/research`	Intelligence feed	RSS, Reddit, optional X, entity/ticker extraction, sentiment, source controls
`/documents`	Secure vault	Upload, OCR, metadata, retention, links to transactions and tasks
`/forms`	Preparation workspace	Form templates, extracted fields, review checklist, official-export links
`/chat`	Financial analyst	Grounded questions, cited calculations, scenario analysis
`/alerts`	Alert center	Rules, channels, delivery log, mute/escalation settings
`/settings/connections`	Integration controls	Plaid, Reddit, market data, RSS, document settings
`/settings/security`	Privacy/security	Sessions, encryption state, exports, deletion, audit events

Technical Stack

Layer	V1 selection	Why
Web app	Next.js 15+, TypeScript, Tailwind, shadcn/ui	Matches your existing product workflow and Vercel deployment
Charts	Recharts or Tremor	Fast, accessible financial visualizations
Hosting	Vercel	Preview deployments, secure environment variables, cron triggers
Database/Auth/Storage	Supabase Postgres, Auth, Storage, Realtime	Strong fit for relational ledgers and controlled files
Authorization	Supabase RLS + server-only service role	RLS must be enabled for exposed schemas; it pairs with Supabase Auth for user-scoped authorization. [17]
Queue/worker	Trigger.dev, Inngest, or BullMQ + Upstash Redis	Durable webhook and OCR processing

Layer	V1 selection	Why
Banking	Plaid Link, Transactions, Investments, webhooks	Existing user access and read-only aggregation
Markets	Alpaca Market Data, SEC EDGAR, RSS	Free/low-cost development path
Social	Reddit OAuth; optional X API	Explicitly opt-in and cost-contained
LLM	OpenAI/Anthropic via Vercel AI SDK	Tool calling, structured outputs, streaming
Document extraction	OCR provider adapter + local PDF text extraction	Keep provider swappable; no fixed vendor lock-in
Notifications	Resend email, web push, optional SMS adapter	Alert delivery with audit trail
Observability	Sentry, Vercel logs, structured audit tables	Trace failures without leaking financial data
IaC / CI	GitHub Actions, Vercel, Supabase migrations	Reproducible deployment and schema governance

Key Sheet

Definitions

Term	Definition
Working capital	Available liquid funds less short-term obligations, shown separately from investments
Posted transaction	A settled bank transaction
Pending transaction	An authorized but not settled transaction; excluded from finalized P&L by default
Ledger transaction	Normalized internal financial record derived from one or more external records
Transfer	Internal movement between owned accounts; excluded from income/expense P&L
Allocation	User-defined personal, business, reimbursable, tax-relevant, or shared percentage
Financial close	Review workflow that locks a monthly P&L snapshot while retaining correction history
Signal	A sourced market/social observation, never a recommendation
Research confidence	Evidence quality score based on primary source, corroboration, recency, and extraction certainty

Naming Conventions

```
Database: snake_case plural tables
TypeScript: PascalCase types, camelCase values/functions
External IDs: provider_resource_id (e.g., plaid_transaction_id)
Internal IDs: UUID v7
Event IDs: provider:event_type:provider_event_id
```

```
Money: integer minor units + ISO currency code, never floating point
Time: timestamptz in UTC; render America/Chicago in UI
```

Required Environment Variables

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

PLAID_CLIENT_ID=
PLAID_SECRET=
PLAID_ENV=sandbox
PLAID_WEBHOOK_SECRET=

ALPACA_API_KEY=
ALPACA_API_SECRET=
SEC_USER_AGENT="CapitalCommand contact@yourdomain.com"

REDDIT_CLIENT_ID=
REDDIT_CLIENT_SECRET=
REDDIT_REDIRECT_URI=

OPENAI_API_KEY=               # or ANTHROPIC_API_KEY
ENCRYPTION_MASTER_KEY=
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
RESEND_API_KEY=
SENTRY_DSN=
```

Core Data Model

```
profiles
financial_connections
financial_accounts
plaid_items
raw_provider_events
transactions_raw
ledger_transactions
transaction_splits
merchant_rules
categories
budgets
budget_periods
goals
goal_contributions
recurring_obligations
monthly_closes
pnl_lines
portfolio_accounts
securities
positions_snapshots
investment_transactions
market_instruments
```

```
market_quotes
watchlists
research_sources
research_items
research_entities
documents
document_chunks
form_templates
form_instances
alerts
alert_deliveries
chat_threads
chat_messages
chat_citations
audit_logs
```

Critical Ledger Rules

- `transactions_raw` is immutable except for provider correction metadata.
- `ledger_transactions` stores normalized business meaning and can be superseded but never destructively edited.
- A P&L is generated from `ledger_transactions`, not raw bank records.
- Every user override includes `changed_by`, `changed_at`, previous value, reason, and optional review status.
- Transfers are detected by amount, timing, account ownership, and user confirmation—never only by merchant name.
- Tax categories are labels for preparation, **not** a determination that an expense is deductible.

Jobs To Be Done

Role	Job	Inputs	Outputs	ROSTR agent
Financial Data Agent	Import and normalize financial activity	Plaid webhook, sync cursor, account metadata	Ledger candidates, balances, event audit	`financial-sync-agent`
Ledger Steward	Categorize and reconcile activity	Raw transaction, rules, user corrections	Classified ledger transaction	`ledger-agent`
Cash Forecaster	Explain liquidity and expected obligations	Balances, recurring items, goals, transactions	7/30/90-day forecast	`cash-flow-agent`
P&L Controller	Produce reviewable monthly P&L	Allocated ledger, close policy	P&L statement and exception queue	`pnl-agent`
Portfolio Observer	Track positions and exposure	Plaid/portfolio data, prices, watchlists	Allocation and exposure metrics	`portfolio-agent`

Role	Job	Inputs	Outputs	ROSTR agent
Market Intelligence Agent	Convert sources into cited research	Market APIs, EDGAR, RSS, Reddit, X if enabled	Ranked signal cards, no trade commands	`research-agent`
Document Steward	Store/extract/tag documents	Upload, OCR text, metadata	Searchable encrypted document record	`document-agent`
Financial Analyst	Answer grounded questions	Authorized retrieved context	Cited answer, calculation, uncertainty	`finance-chat-agent`
Risk Guardian	Enforce permissions and safety	Request context, action policy	Allow/deny/audit decision	`policy-agent`

Use Cases

Happy Paths

1. **Bank connection:** User clicks “Connect account” → Plaid Link → server exchanges token → stores encrypted token → initial sync begins → dashboard shows import status → transactions arrive.
2. **Daily alert:** A Plaid webhook signals changes → worker syncs deltas → a new transaction exceeds a configured threshold → alert is created → email/push is sent → delivery outcome is logged.
3. **Working capital review:** User opens Command Center → sees posted cash, pending amount, 30-day obligations, forecast low point, and top cash-flow drivers.
4. **Monthly P&L:** User reviews uncategorized and split transactions → confirms business allocations → generates draft P&L → approves month close → snapshot is retained.
5. **Investment research:** User adds symbols → system pulls price/market data, SEC sources, and opted-in social/RSS sources → analyzer produces a cited research brief with sentiment separated from facts.
6. **Chat:** User asks, “Why is my runway down this month?” → retrieval gathers ledger deltas and recurring obligations → agent calculates answer → cites source rows and assumptions.

Edge Cases and Fallbacks

Event	Required behavior	State change
Duplicate webhook	Deduplicate by provider event ID and payload hash	Event marked `duplicate_ignored`
Plaid sync cursor invalid	Re-run controlled full sync; do not duplicate normalized records	Connection marked `recovery_sync`
Account disconnected	Show reconnect CTA; preserve historical data	Connection `requires_reauth`
Pending becomes posted	Link records; do not double-count expense	Transaction lifecycle updated
Merchant/category uncertain	Assign `needs_review`; do not create a permanent rule	Review queue item created

Event	Required behavior	State change
Market quote unavailable	Show last-known timestamp and stale state	Quote `stale=true`
Social source has no evidence	Label as “community signal”; do not elevate to fact	Confidence capped
OCR failure	Preserve original document; request manual metadata	Document `needs_review`
LLM lacks evidence	Answer “I don’t have sufficient data,” then offer next retrieval/action	Response includes gap
User requests trade execution	Refuse execution; offer educational analysis and risk framing	Audit `prohibited_action`

Custom Scripts and Templates

Plaid Webhook Intake Schema

```
type PlaidWebhookEnvelope = {
  webhook_type: string;
  webhook_code: string;
  item_id: string;
  new_transactions?: number;
  removed_transactions?: string[];
  error?: { error_code: string; error_message: string };
};
```

LLM Tool Contract

```
{
  "name": "query_financial_ledger",
  "description": "Retrieve user-authorized, normalized ledger data for a bounded date
  "parameters": {
    "type": "object",
    "properties": {
      "start_date": { "type": "string", "format": "date" },
      "end_date": { "type": "string", "format": "date" },
      "account_ids": { "type": "array", "items": { "type": "string" } },
      "categories": { "type": "array", "items": { "type": "string" } },
      "include_pending": { "type": "boolean", "default": false },
      "purpose": { "type": "string" }
    },
    "required": ["start_date", "end_date", "purpose"]
  }
}
```

Repository Structure

```
capital-command/
├── apps/
│   └── web/
│       ├── app/(dashboard)/
│       ├── app/api/
│       ├── components/
│       └── lib/
├── packages/
│   ├── db/
│   ├── finance-domain/
│   ├── provider-adapters/
│   ├── ai-policy/
│   └── ui/
├── workers/
│   ├── plaid-sync/
│   ├── alerts/
│   ├── document-processing/
│   └── research-ingestion/
├── supabase/
│   ├── migrations/
│   ├── seed.sql
│   └── policies.sql
├── docs/
│   ├── threat-model.md
│   ├── accounting-policy.md
│   └── runbooks/
└── .context-engine/sessions/capital-command/
```

Integrations and Permission Model

Integration	V1 scope	Permission policy
Plaid	Read transactions, balances, investments; webhook notifications	No ACH, transfer, payment initiation, or account-verification actions in V1
Alpaca Market Data	Read market data only	Do not connect trading API/order endpoints
SEC EDGAR	Public filing/financial facts read-only	Identify source and timestamp
Reddit	Read selected communities/searches through OAuth	No posting, voting, DMs, or account modification
X	Optional read-only recent search/stream	Disabled by default; no posting or engagement
Supabase	User-scoped database and file storage	RLS on all exposed tables; service role restricted to server/workers
LLM provider	Structured retrieval and analysis	No direct database access; tools enforce user scope and read-only data
Documents	Upload/read/delete by owner	Encryption, malware scan, retention and export/delete controls

Integration	V1 scope	Permission policy
IRS/form sources	Link/export guidance only	No autonomous filing or tax representation

RAG DAL Findings

- **Tier 1 — Plaid:** Use incremental transaction sync and webhooks instead of polling every account; this is the data path for daily activity. Confidence: 0.95. [5] [4]
- **Tier 1 — Plaid:** Investment holdings and investment transactions are available for supported investment accounts; model these separately from depository transactions. Confidence: 0.90. [7] [6]
- **Tier 1 — SEC:** SEC EDGAR provides public JSON endpoints for filing histories and XBRL company facts; use it as the primary fundamentals source. Confidence: 0.95. [11] [10]
- **Tier 1 — Supabase:** Enforce Row Level Security on all exposed financial tables. Confidence: 0.95. [17]
- **Tier 2/3 — Social:** Reddit can support controlled personal research ingestion, but social content must remain a low-confidence signal. X should be an optional paid adapter, not a V1 dependency. Confidence: 0.85. [12] [13]

PAL Compilation Summary

```
PAL_Compilation_Summary:
  L1 — Intent:
    Goal: Build a private financial operating dashboard for working capital,
      investments, documents, and grounded AI analysis.
    Role: Capital Command orchestration system.
    Constraints: Read-only V1; free/low-cost APIs; Plaid available; no trade
      execution, tax filing, or legal advice.
    4Ds Phase: PreD transitioning to D1.
    CoE Gap: Partially — no locked accounting, security, or data-retention policy.
    API Readiness: Partial — Plaid ready; market/social/document provider decisions d

  L2 — Composition:
    N: security model, ledger schema, Plaid production validation, accounting policy
    A: real-time definition, P&L entity allocation, market-data entitlements, X scope
    P: dashboard, sync worker, budgets/goals, P&L, portfolio, documents, chat.
    O: social streaming, advanced scenarios, broker execution, multi-user.
    Execution: N → A → P → O.

  L3 — Optimization:
    Context: long_term ContextEngine; sampling balanced, deterministic for calculatio
    P1 tools: Plaid, Supabase, Vercel, Alpaca data, SEC EDGAR, LLM retrieval.
    P2 tools: Reddit OAuth, RSS, document OCR.
    P3 tools: X API, SMS, tax/form adapters.
    Risks: incorrect categorization, sensitive documents, financial hallucinations,
      paid vendor scope creep.

  L4 — Compilation:
    System prompt ✓; Capital Command ROSTR DSL manifest ✓.
```

```
  L5 — Runtime:
    NPAO canvas: 6 N / 6 A / 11 P / 5 O tasks.
    Handoff: Builder-ready after D1 gates pass.
```

PAL Agent Spec

```
agent:
  name: "Capital Command Orchestrator"
  version: "0.1.0"
  phase: "D1"
  objective: |
    Maintain a private, read-only financial operating system that reconciles
    user-authorized financial data, market research, and documents into
    explainable dashboards and grounded AI analysis.
  inputs:
    - authenticated_user_id: uuid (required)
    - financial_connection_events: object (optional)
    - user_queries: string (optional)
    - documents: array (optional)
    - watchlists: array (optional)
  outputs:
    type: object
    properties:
      financial_state:
        type: object
        format: json
      alerts:
        type: array
        format: json
      grounded_analysis:
        type: string
        format: markdown
    required: [financial_state]
  tools:
    - rag_dal.web_search
    - plaid_adapter.read
    - market_data_adapter.read
    - sec_edgar_adapter.read
    - finance_ledger.query
    - document_store.query
  memory:
    type: "long_term"
    backend: "context_engine"
    storage_path: ".context-engine/sessions/capital-command/"
  guardrails:
    - "Never execute trades, transfers, filings, or legal actions."
    - "Never expose raw tokens, account numbers, or full sensitive document text."
    - "Every numerical financial answer must cite internal records."
    - "Separate fact, calculation, forecast, and community signal."
    - "Require explicit user approval for any future write-capable integration."
    - max_tool_calls: 12
    - timeout_seconds: 90
```

3. Project Architecture Diagram

```
flowchart TB
  U[Authenticated User] --> UI[Next.js / Vercel UI]
  UI --> AUTH[Supabase Auth]
  AUTH --> RLS[Supabase RLS Policies]

  subgraph UX["Application Surface"]
    DASH[Command Center]
    LEDGER[Transactions / P&L / Budgets / Goals]
    PORT[Portfolio / Markets]
    RES[Research Feed]
    DOCS[Documents / Forms]
    CHAT[Grounded Financial Chat]
  end

  UI --> UX

  subgraph PAL["PAL Compilation Nodes"]
    L1[PAL L1 Intent & Constraints]
    L2[PAL L2 Functional Composition]
    L3[PAL L3 Tool + Risk Optimization]
    L4[PAL L4 Agent Manifests]
    L5[PAL L5 Runtime Queue]
    L1 --> L2 --> L3 --> L4 --> L5
  end

  subgraph NPAO["NPAO Decision Path"]
    N[N: Security / Schema / Auth]
    A[A: Policy Ambiguities / Edge Cases]
    P[P: Product Features]
    O[O: Advanced Signals / Automation]
    N --> A --> P --> O
  end

  PAL --> NPAO
  NPAO --> HUB

  subgraph HUB["ROSTR Hub"]
    RT[Runtime: versioned agent configs]
    ORCH[Orchestration: sequential + conditional]
    STATE[State: ContextEngine + audit state]
    TOOLS[Tools: permissioned adapters]
    REF[Reference: policies, accounting rules, RAG sources]
  end

  subgraph EXT["External Providers"]
    PLAID[Plaid: Link / Transactions / Investments]
    ALPACA[Alpaca: Market Data]
    SEC[SEC EDGAR]
    REDDIT[Reddit OAuth]
    X[X API Optional]
    RSS[RSS Sources]
    OCR[OCR / Document Parser]
  end
```

```
PLAID -->|webhook| INBOX[Raw Provider Event Inbox]
EXT --> INBOX
INBOX --> QUEUE[Durable Job Queue]
QUEUE --> WORKER[Sync / Normalize / Extract Workers]

WORKER --> DAL
subgraph DAL["RAG DAL Execution Layer"]
  FIN[Financial data adapter]
  MKT[Market data adapter]
  DOC[Document retrieval adapter]
  WEB[SEC / RSS / Reddit / X adapters]
  RET[Source ranking + provenance]
  FIN --> RET
  MKT --> RET
  DOC --> RET
  WEB --> RET
end

DAL --> DB[(Supabase Postgres\nLedger + Metrics + Policies)]
WORKER --> STORE[(Supabase Storage\nEncrypted Documents)]
DB --> METRICS[Materialized Cash / P&L / Goal / Portfolio Metrics]
METRICS --> ALERTS[Alert Evaluation Engine]
ALERTS --> NOTIFY[Email / Push Notification Adapter]

CHAT --> POLICY[Policy Agent]
POLICY --> RETRIEVE[Scoped Retrieval + Calculations]
RETRIEVE --> DB
RETRIEVE --> STORE
RETRIEVE --> DAL
RETRIEVE --> LLM[LLM with Structured Tools]
LLM --> CITED[Cited Answer + Uncertainty Labels]
CITED --> CHAT

DB --> STATE
STORE --> STATE
STATE --> CTX[.context-engine/sessions/capital-command/]
```

ROSTR Hub State Persistence

- **Runtime:** versioned prompts, policies, feature flags, and provider adapters.
- **Orchestration:** Plaid webhooks and uploads enter a durable queue; chat requests stay synchronous but use bounded read-only retrieval.
- **State:** sync cursors, transaction mappings, user corrections, alert delivery status, document metadata, and source provenance persist in Postgres; project decisions persist in ContextEngine.
- **Reference:** accounting policy, category taxonomy, LLM guardrails, data retention policy, and research-source credibility rules are versioned in Git.

The Hub mapping uses the framework's Runtime, Orchestration, State, Tools, and Reference layers; ContextEngine stores project-level session artifacts under `.context-engine/sessions/[project-name]/`.[18]

4. Workflow Architecture Plan

PreD — Feasibility Research

Necessity

- **N1. Confirm Plaid product access and live-data limits** | Owner: Platform engineer | Done when: Sandbox connection, webhook receiver, Transactions sync, and Investments availability are verified against your Plaid account. Plaid offers free Sandbox use, while production is governed by plan-specific limits/pricing. [1]
- **N2. Lock V1 accounting policy** | Owner: Product owner + accountant review | Done when: transfer, reimbursement, owner draw, business allocation, and taxable-category rules are documented.
- **N3. Complete threat model** | Owner: Security engineer | Done when: secrets, PII, document data, webhook replay, account takeover, and tenant-isolation controls are documented.
- **N4. Define "real-time" SLO** | Owner: Product owner | Done when: UI language states "provider update + processing target," not a guaranteed instant bank feed.
- **N5. Confirm document retention and deletion rules** | Owner: Product owner | Done when: encrypted storage, export, deletion, and backup retention behavior are approved.
- **N6. Validate market-data entitlements** | Owner: Platform engineer | Done when: free-development feed coverage, latency labels, and options data constraints are visible in product copy.

Anxiety

- **A1. Separate personal, business, and reimbursable activity** | Done when: each transaction supports split allocations and review workflow.
- **A2. Decide whether X belongs in V1** | Recommendation: no. Ship RSS + SEC + Reddit first; make X an adapter after pricing and use-case validation.
- **A3. Define P&L semantics** | Recommendation: cash-basis personal/business operating P&L, not GAAP financial statements.
- **A4. Define research-signal policy** | Done when: social sentiment is labeled as non-verified signal; all recommendations are prohibited.
- **A5. Choose OCR/document processor** | Done when: PII handling, retention, cost, and extraction accuracy are tested with representative redacted files.
- **A6. Establish AI data boundaries** | Done when: user can opt out of document/financial retrieval per chat and all answers require sources.

PreD Go/No-Go: Go for a private, read-only V1. **No-go** for autonomous trading, tax filing, or unsupervised legal paperwork.

Design — Blueprinting

Priority

1. Create Supabase schema, RLS policies, audit tables, and migrations.
2. Build provider adapter interfaces: `BankDataProvider`, `MarketDataProvider`, `ResearchProvider`, `DocumentExtractor`.
3. Define Plaid event idempotency, cursor persistence, connection health state machine, and re-auth flow.
4. Lock category hierarchy, business-allocation model, transfer matching rules, and monthly-close workflow.
5. Wireframe the Command Center, Ledger, Budget, Goals, P&L, Portfolio, Research, Documents, and Chat routes.
6. Define alert grammar: trigger, condition, threshold, schedule, channel, cooldown, deduplication key.
7. Define LLM tool schemas and citation payloads.
8. Create policy tests that prove the assistant cannot perform transfers, trades, filings, or unsourced financial claims.

D1 exit gate: No D2 development until N1–N6 and A1–A6 are resolved and reviewed.

Development — Implementation and Testing

Priority

1. Build auth, profile, onboarding, connection settings, and encrypted secret storage.
2. Implement Plaid Link, token exchange, initial sync, webhook intake, queue processing, and transaction normalization.
3. Build transaction ledger and human-in-the-loop categorization/rules.
4. Build Command Center and cash-flow forecast.
5. Build budget, goal, recurring obligation, and P&L modules.
6. Implement market watchlists, quotes, SEC card ingestion, and portfolio display.
7. Implement RSS and Reddit ingestion; defer X.
8. Build encrypted document vault, extraction jobs, metadata/tagging, and transaction linking.
9. Implement retrieval-first LLM chat with citations and calculation tools.
10. Add alerts, notification delivery logs, retries, and user controls.
11. Add test suites: unit, integration, webhook replay, RLS, prompt-injection, and end-to-end Plaid Sandbox tests.

Development Acceptance Criteria

- No financial token reaches the browser.
- All financial tables enforce user-scoped RLS.
- A duplicate webhook cannot duplicate a ledger transaction or alert.
- A deleted/revoked connection no longer syncs, but historical ledger records remain traceable.
- Every LLM monetary answer lists its date range and data sources.
- Research feed cards expose source, timestamp, confidence, and content class.
- All user-generated document uploads have malware validation and access logging.

Deployment — Safe Production Release

Necessity

1. Deploy staging on Vercel with a separate Supabase project and Plaid Sandbox.
2. Run a privacy/security review and tabletop webhook-replay incident.
3. Run a 30-day shadow ledger: compare dashboard totals against source statements.
4. Enable production Plaid only after webhook and reconnect flows are proven.
5. Require explicit approval before production deployment, as the ROSTR D3 gate requires.

Release Sequence

- Internal private alpha: one user, two depository accounts, no document uploads.
- Private beta: add budgets, goals, P&L, and alerting.
- Research beta: add markets, SEC, RSS, and Reddit.
- Secure document beta: add vault and form-preparation workspace after security validation.
- Optional advanced beta: X source, brokerage read-only, multi-entity financial views.

Debugging — Defect Resolution

Anxiety / Opportunity

- Monitor sync lag, transaction duplicates, stale quotes, categorization overrides, failed alerts, RLS denials, document extraction failures, and LLM citation coverage.
- Treat any cross-user data exposure, token exposure, duplicate ledger posting, or unauthorized provider action as a **Severity 1** incident.
- Maintain runbooks for Plaid degraded state, Supabase outage, webhook signature failures, document-processing backlog, and LLM provider outage.

5. LLM Build Prompts

Master Orchestration Prompt

```
You are the Capital Command Master Builder operating under the ROSTR framework.

Mission:
Build a private, read-only personal financial operating system that consolidates
Plaid-authorized bank and investment data, budgets, goals, P&L, market research,
documents, and grounded LLM analysis into a secure Next.js + Supabase dashboard.

Mandatory framework behavior:
1. Run PAL stages L1 through L5 in order. Label stated facts [Extracted],
   justified defaults [Inferred], and unknowns [UNKNOWN — enrich via RAG DAL].
2. Apply NPAO sequencing: Necessity → Anxiety → Priority → Opportunity.
3. Do not advance from D1 Design to D2 Development until all design gates are locked
4. Use RAG DAL provenance: Tier 1 official sources for provider/API decisions;
   Tier 3 sources such as Reddit/X are community signals only.
5. Persist project decisions and artifact references to:
   .context-engine/sessions/capital-command/

Technical constraints:
- Next.js, TypeScript, Vercel, Supabase Auth/Postgres/Storage/RLS.
- Plaid is read-only: Transactions, Investments, Balance, Link, and webhooks.
- Store Plaid tokens only server-side, encrypted at rest.
- Implement append-only raw events, idempotent sync jobs, normalized ledger records,
  audited user overrides, and materialized financial metrics.
- Money must use integer minor units and ISO currency codes, never floats.
- Every external event requires deduplication and replay-safe processing.
- All exposed Supabase tables require RLS.
- All LLM tools are read-only and must enforce authenticated user scope.
- Every materially financial chat response must return calculation date range,
  cited data records, assumptions, uncertainty, and a non-advisory notice.
- Never execute trades, transfers, tax filing, document signing, or legal actions.
- Never imply real-time data unless it is timestamped and provider freshness is shown

Build output:
1. Repository tree.
2. Supabase migrations and RLS policies.
3. Provider-adapter interfaces.
4. Plaid Link/token/webhook/sync implementation.
5. Dashboard routes and component plan.
6. Alert rule engine.
7. Grounded LLM tool schemas and evaluation tests.
8. Security threat model and production checklist.
9. Test plan with acceptance criteria.

Quality bar:
A single user must be able to connect an account, see normalized daily
transactions, understand working capital, manage a budget and goal, review a
draft P&L, inspect a watchlist, upload a document, and ask a cited financial
question—without any write-capable financial action.
```

Financial Sync Agent

```
You are the Financial Sync Agent for Capital Command.

Objective:
Ingest Plaid-authorized financial data safely and convert it into immutable raw
events plus reviewable normalized ledger candidates.

Inputs:
- Plaid webhook payload
- encrypted item reference
- persisted transactions/sync cursor
- account ownership and user preferences

Procedure:
1. Verify webhook authenticity and create an idempotency key.
2. Store the raw provider payload before transformation.
3. On SYNC_UPDATES_AVAILABLE, run incremental transactions sync until has_more=false
4. Upsert raw transaction records by provider transaction ID.
5. Detect added, modified, and removed records without destructive ledger deletion.
6. Produce normalized ledger candidates with account, amount_minor, currency,
   posted_at, pending state, merchant, category confidence, and source lineage.
7. Route uncertain transfer, reimbursement, and category decisions to review.
8. Recalculate only affected balances, forecasts, budgets, and alerts.

Guardrails:
- Never expose or log access tokens.
- Never use floats for money.
- Never classify a transfer as income/expense without confidence or user review.
- Never create duplicate transactions from webhook retries.
- Return structured result: event status, sync stats, errors, review queue, and cita
```

Ledger and P&L Agent

```
You are the Ledger and P&L Agent for Capital Command.

Objective:
Maintain an explainable cash-basis operating ledger and generate a reviewable
monthly P&L; do not provide accounting, tax, or legal advice.

Rules:
- Raw provider transactions are immutable.
- Normalization, categories, splits, business allocations, and reimbursements are
  versioned user decisions.
- Exclude confirmed internal transfers from income/expense P&L.
- Separate posted and pending activity.
- Use the selected reporting entity and month-close policy.
- Flag ambiguous classifications instead of guessing.
- Show all totals in integer-minor-unit-backed currency formatting.

Output:
- P&L lines by income and expense category
- personal/business/reimbursable allocation summary
- uncategorized and exception queue
```

```
- comparison to prior month and budget
- source transaction citations
- assumptions and review required fields
```

Market Intelligence Agent

```
You are the Market Intelligence Agent for Capital Command.

Objective:
Create source-cited, educational market briefs for user watchlists across equities,
options, crypto, energy, and commodity proxies. You do not issue trade signals,
execute orders, personalize investment advice, or make price predictions.

Source hierarchy:
1. Official market/provider data and SEC filings
2. Reputable reporting and issuer materials
3. RSS and authenticated Reddit/X community content, labeled Community Signal

For each item:
- Separate price/market fact, SEC/company fact, analyst calculation, and community se
- Include source URL/identifier, timestamp, recency, and confidence.
- Explain uncertainty, material data gaps, and options risks.
- For options, state that contracts can lose their full premium and that short/uncove
  positions carry additional risk; do not suggest strikes, expirations, or execution
- Never treat social sentiment as verified information.

Return:
- market snapshot
- catalyst calendar
- filing/news evidence
- community-signal digest
- risk flags
- questions the user should research next
```

Document and Form Agent

```
You are the Document and Form Agent for Capital Command.

Objective:
Securely extract, organize, and prepare user documents for review without providing
legal advice, tax advice, or autonomous filing/signing.

Procedure:
1. Accept only authorized user uploads.
2. Preserve original file, create encrypted derivative text, and record extraction co
3. Classify document type, entities, dates, amounts, deadlines, and linked transactio
4. Populate a reviewable field map for approved templates.
5. Identify missing fields and contradictions.
6. Present official source links or export-ready data; never submit, sign, or file an

Guardrails:
- Minimize display of SSNs, account numbers, addresses, and signatures.
- Require user review for every extracted value.
```

```
- Mark inferred fields as unverified.
- State clearly: “This is organizational assistance, not tax or legal advice.”
```

Grounded Finance Chat Agent

```
You are the Grounded Finance Chat Agent for Capital Command.

Before answering:
1. Classify the request as ledger fact, budget/goal analysis, P&L analysis,
   document lookup, market research, scenario analysis, or prohibited action.
2. Retrieve only user-authorized data needed for the answer.
3. Perform calculations deterministically using tools, not model arithmetic.
4. Cite internal records and external sources.
5. Distinguish observed facts, calculations, forecasts, and community signals.

Required response format:
- Direct answer
- Evidence: data range, records/sources used
- Calculation or assumptions
- Confidence / gaps
- Next safe action

Hard refusals:
- Execute a trade, transfer, tax filing, or document signature
- Provide unsourced account balances or transaction claims
- Reveal raw tokens, full sensitive IDs, or another user’s data
- Present research/social content as personalized investment advice
```

This is a technically feasible build, but its quality depends on treating it first as a **secure ledger and decision-support system**, not as a trading or tax-filing app. The best V1 is Plaid transactions + cash flow + budget/goals + P&L + grounded chat, with market research and document workflows added behind strict permission and provenance controls.

⁂

1. https://plaid.com/docs/account/billing/
2. https://www.finra.org/rules-guidance/notices/22-08
3. https://www.finra.org/investors/investing/investment-products/options
4. https://plaid.com/docs/api/products/transactions/
5. https://plaid.com/docs/transactions/
6. https://plaid.com/docs/api/products/investments/
7. https://plaid.com/docs/investments/
8. https://docs.alpaca.markets/us/docs/about-market-data-api
9. https://docs.alpaca.markets/us/docs/options-trading
10. https://www.sec.gov/search-filings/edgar-application-programming-interfaces
11. https://www.sec.gov/about/developer-resources
12. https://support.reddithelp.com/hc/en-us/articles/16160319875092-Reddit-Data-API-Wiki

13. https://docs.x.com/x-api/posts/filtered-stream/introduction
14. https://docs.x.com/x-api/posts/search/introduction
15. https://www.irs.gov/e-file-providers/free-file-fillable-forms
16. https://www.irs.gov/newsroom/how-to-create-an-account-to-use-irs-free-file-fillable-forms-youtube-video-text-script
17. https://supabase.com/docs/guides/database/postgres/row-level-security
18. ROSTR_HUB_CONTEXTENGINE.md
19. NPAO_4Ds_FRAMEWORK.md
20. PAL_FRAMEWORK.md
21. PRD_TEMPLATE.md
22. PROJECT_INTAKE_QUESTIONNAIRE.md
23. RAGDAL_FRAMEWORK.md
24. ROSTR_BUILD_SPEC.md
25. https://apis.io/apis/sec-edgar/sec-edgar-xbrl-api/
26. https://plaid.com/docs/api/webhooks/
27. https://github.com/alpacahq/alpaca-py
28. https://dlthub.com/context/source/alpaca-markets
29. https://sec-edgar-api.readthedocs.io/
30. https://github.com/plaid/plaid-node/issues/140
31. https://alpaca.markets/
32. https://www.reddit.com/r/redditdev/comments/14nbw6g/updated_rate_limits_going_into_effect_over_the/
33. https://painonsocial.com/blog/reddit-api-limitations
34. https://docs.x.com/x-api/posts/filtered-stream/integrate/build-a-rule
35. https://www.reddit.com/r/redditdev/comments/145liwv/api_changes_and_personal_oauth/
36. https://docs.x.com/x-api/webhooks/stream/introduction
37. https://www.merrilledge.com/investment-products/options/benefits-risks-of-options
38. https://apidog.com/blog/reddit-api-guide/
39. https://twitterapi.io/blog/twitter-streaming-api
40. https://www.lexology.com/library/detail.aspx?g=22666313-0589-4e5c-98a1-8bd5c16d5e4f
41. https://www.reddit.com/r/redditdev/comments/15gwi25/different_ratelimits/
42. https://plaid.com/pricing/
43. https://support.plaid.com/hc/en-us/articles/16194695660311-Can-I-use-Plaid-for-free
44. https://www.irs.gov/e-file-providers/free-file-fillable-forms-program-limitations-and-available-forms
45. https://www.reddit.com/r/Supabase/comments/1ppq2qv/need_clarifications_on_row_level_security/
46. https://www.youtube.com/watch?v=sh-InC2JAM8
47. https://supabase.com/features/row-level-security
48. http://beancount.io/forum/t/working-within-plaid-s-free-tier-what-you-need-to-know/121
49. https://www.irs.gov/forms-instructions

50. https://dev.to/asheeshh/mastering-supabase-rls-row-level-security-as-a-beginner-5175

51. https://www.fintegrationfs.com/post/plaid-api-pricing-structure-and-plans

52. https://www.irs.gov/e-file-do-your-taxes-for-free

53. projects.agent_platform.product_architecture